1 Introduction to cloud native

This chapter covers

- What the cloud and cloud computing model are
- The definition of cloud native
- Characteristics of cloud native applications
- Culture and practices supporting cloud native
- When and why you might consider the cloud native approach
- Topologies and architectures for cloud native applications

Cloud native applications are highly distributed systems that live in the cloud and are resilient to change. Systems are made up of several services that communicate through a network and are deployed in a dynamic environment where everything keeps changing.

Before diving into the technologies, it's fundamental to define what cloud native is. Like other buzzwords in our field (such as *agile*, *DevOps*, or *microservices*), *cloud native* is sometimes misunderstood and can be a source of confusion because it means different things to different people.

In this chapter, I'll provide you with the conceptual tools you'll need for the rest of the book. I'll start by defining what cloud native means and what it takes for an application to be identified as such. I'll explain the properties of cloud native applications, examine the characteristics of the cloud computing model, and discuss when and why you might want to move to the cloud. I'll also present some fundamental concepts of cloud native topologies and architectures. Figure 1.1 shows an overview of the different elements I'll cover in this chapter as I define and qualify cloud native systems. At the end of this chapter, you'll be ready to start your journey building cloud native applications with Spring and deploying them to Kubernetes.

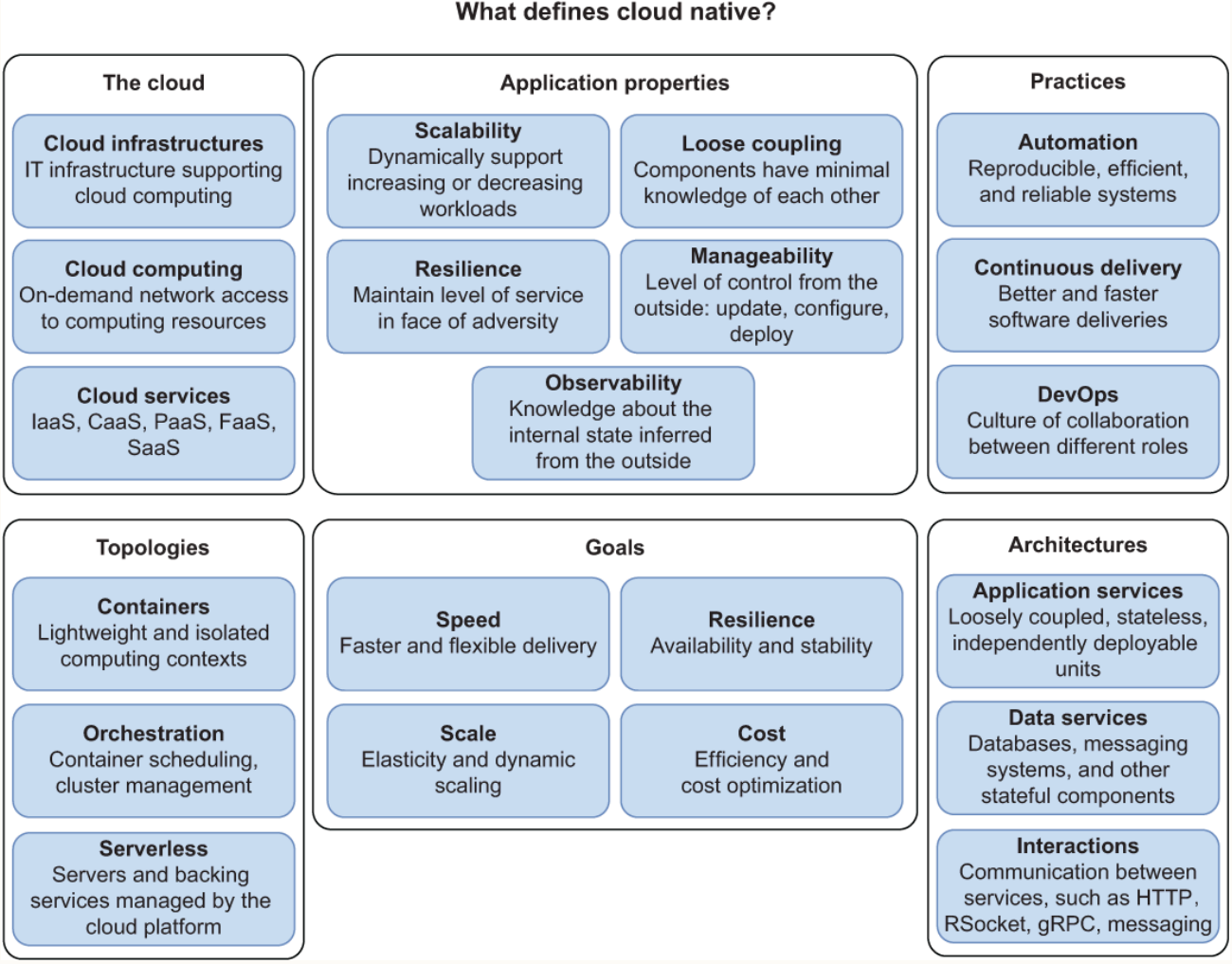


Figure 1.1 Cloud native is an approach to application development aiming at leveraging cloud technologies.

1.1 What is cloud native?

On May 25, 2010, Paul Fremantle, a veteran of the cloud industry, wrote a post on his blog titled "Cloud Native."[1] He was among the first to use the term *cloud native*. At a time when concepts and technologies like microservices, Docker, DevOps, Kubernetes, and Spring Boot didn't yet exist, Fremantle discussed what it takes for "applications and middleware to work well in a cloud environment"—to be *cloud native*—with his team at WSO2.

The key concept explained by Fremantle is that cloud native applications should be specifically designed for the cloud and have properties that take advantage of the cloud environment and the cloud computing model. You can move a traditional application (designed to run *on the ground*) to the cloud, an approach commonly referred to as "lift and shift," but that doesn't make the application *native* to the cloud. Let's see what does.

1.1 The Three Ps of Cloud Native

What does it mean for applications to be designed specifically for the cloud? The Cloud Native Computing Foundation (CNCF) answers that question in its cloud native definition:[2]

Cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.

These techniques enable loosely coupled systems that are resilient, manageable, and observable. Combined with robust automation, they allow engineers to make high-impact changes frequently and predictably with minimal toil.

From this definition, I identify three points that I like to call *The Three Ps of Cloud Native*:

- *Platforms*—Cloud native applications run on platforms based on dynamic, distributed environments: the clouds (public, private, or hybrid).
- *Properties*—Cloud native applications are designed to be scalable, loosely coupled, resilient, manageable, and observable.
- *Practices*—Practices around cloud native applications—automation, continuous delivery, and DevOps—include robust automation combined with frequent and predictable changes.

What is the Cloud Native Computing Foundation?

The Cloud Native Computing Foundation (CNCF) is part of the Linux Foundation, and it "builds sustainable ecosystems and fosters communities to support the growth and health of cloud native open-source software." The CNCF hosts many cloud native technologies and projects to enable cloud portability without vendor lock-in. If you want to discover the many projects addressing any cloud native aspect, I recommend checking out the CNCF Cloud Native Interactive Landscape.[a]

[a] Cloud Native Computing Foundation, "CNCF Cloud Native Interactive Landscape," **https://landscape.cncf.io/**.

In the following sections I'll examine those concepts further. However, I'd first like you to notice how the definition of cloud native is not tied to any specific implementation

detail or technology. The CNCF mentions some in its definition, like containers and microservices, but they are just examples. One of the common misconceptions when starting a migration to the cloud is that you have to adopt a microservices architecture, build containers, and deploy them to Kubernetes. That is not true. Fremantle's post in 2010 is proof of that. He didn't mention any of those because they didn't exist. Yet, the applications he described not only are still considered cloud native, but they also comply with the definition given by the CNCF eight years later.

1.2 The cloud and the cloud computing model

Before focusing on the main characters, the cloud native applications, I'd like to set the scene by describing the context in which cloud native applications run: the *cloud* (figure 1.2). In this section, I'll define the cloud and its main characteristics. After all, if cloud native applications are designed to work well in a cloud environment, we should know what kind of environment that is.

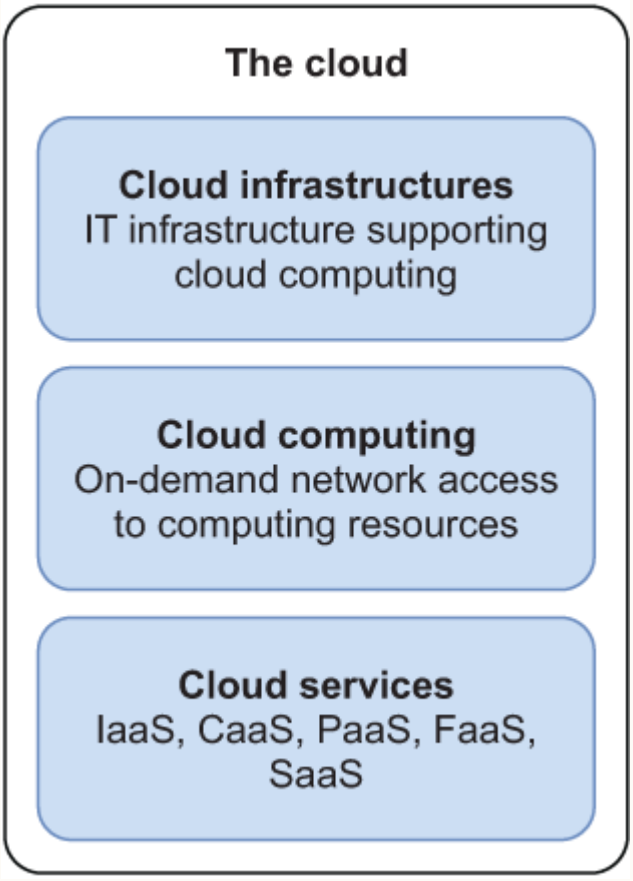


Figure 1.2 The cloud is an IT infrastructure characterized by different computing models and offered as a service by providers according to the degree of control consumers need.

The cloud is an IT infrastructure that supports the delivery of computing resources to consumers according to the cloud computing model. The National Institute of Standards and Technology (NIST) defines cloud computing as follows:[3]

Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

Just like you get electricity from a provider rather than generating it on your own, with the cloud you can get computing resources (such as servers, storage, and networks) as a commodity.

The cloud provider manages the underlying cloud infrastructure, so the consumer doesn't need to worry about physical resources like machines or networks. Companies moving to the cloud can get all the computing resources they need via a network (usually the internet) through a set of APIs that allows them to provision and scale resources as they need on an on-demand, self-service basis.

Elasticity is one of the main characteristics of this model: computing resources can be provisioned and released dynamically, depending on the demand.

Elasticity is the degree to which a system is able to adapt to workload changes by provisioning and de-provisioning resources in an autonomic manner, such that at each point in time the available resources match the current demand as closely as possible.[4]

Traditional IT infrastructures weren't able to provide elasticity. Companies had to calculate the maximum computing capabilities needed and set up an infrastructure that would support that, even if most of it was only required sometimes. With the cloud computing model, the usage of computing resources is monitored, and consumers only pay for what they actually use.

There is no strict requirement about where the cloud infrastructure should be or who should manage it. There are several deployment models for delivering cloud services. The main ones are private cloud, public cloud, and hybrid cloud.

- *Private cloud*—Cloud infrastructure provisioned to be used by a single organization. It can be managed by the organization itself or by a third party, and it can be hosted on premises or externally. A private cloud is usually the preferred option for organizations dealing with sensitive data or highly critical systems. It is also a common choice for having complete control over the infrastructure's compliance with specific laws and requirements like the General Data Protection Regulation (GDPR) or the California Consumer Privacy Act (CCPA). For example, banks and healthcare providers are likely to set up their own cloud infrastructure.

- *Public cloud*—Cloud infrastructure provisioned for public use. It is usually owned and managed by an organization, the *cloud provider*, and is hosted on the provider's premises. Examples of public cloud service providers are Amazon Web Services (AWS), Microsoft Azure, Google Cloud, Alibaba Cloud, and DigitalOcean.
- *Hybrid cloud*—Composition of two or more distinct cloud infrastructures belonging to any of the previous types, bound together and offering services as if they were one single environment.

Figure 1.3 describes the five leading cloud computing service models, what is provided by the platform in each model, and which abstractions are provided to the consumer. For example, with the Infrastructure as a Service (IaaS) model, the platform provides and manages computing, storage, and networking resources, whereas the consumer provisions and manages virtual machines. The decision about which service model to choose should be driven by the degree of control the consumer needs over the infrastructure and which type of computing resources they need to manage.

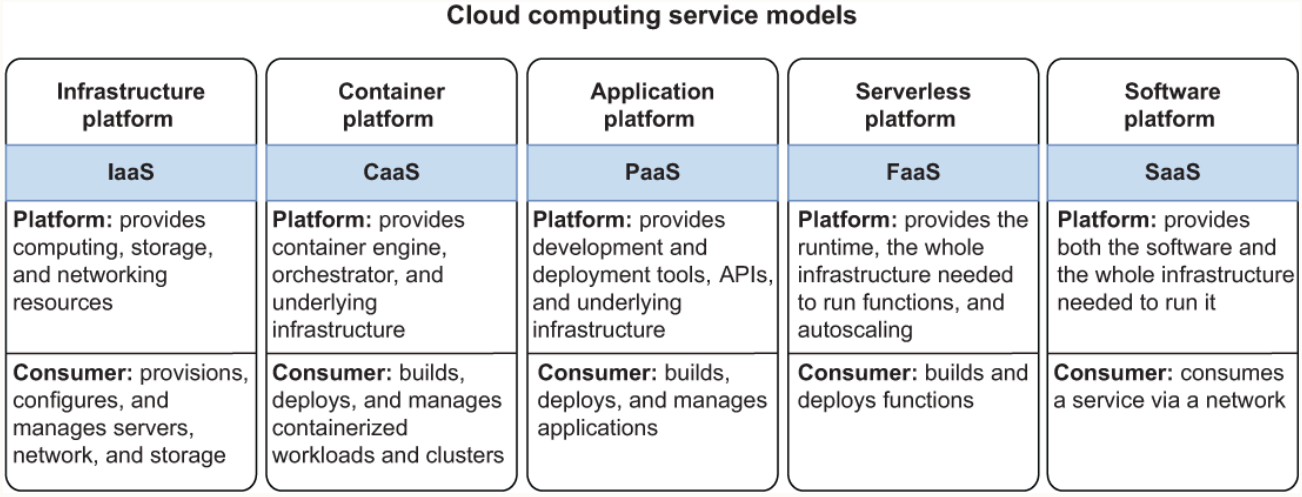


Figure 1.3 The cloud computing service models differ by the level of abstraction they provide and who is responsible for managing which levels (the platform or the consumer).

2.1 Infrastructure as a Service (IaaS)

In the *Infrastructure as a Service* (IaaS) model, consumers can directly control and provision resources like servers, storage, and networks. For example, they can provision virtual machines and install software like operating systems and libraries. Even though this model has been used for a while, it was in 2006 that Amazon made it popular and widely accessible with Amazon Web Services (AWS). Examples of IaaS offerings are AWS Elastic Compute Cloud (EC2), Azure Virtual Machines, Google Compute Engine, Alibaba Virtual Machines, and DigitalOcean Droplets.

2.2 Container as a Service (CaaS)

Using the *Container as a Service* (CaaS) model, consumers cannot control primitive virtualization resources. Instead, they provision and manage containers. The cloud provider takes care of provisioning the underlying resources that fulfill the needs of those containers, such as by starting new virtual machines and configuring networks to make them accessible through the internet. Docker Swarm, Apache Mesos, and Kubernetes are examples of tools used to build container platforms. All major cloud providers offer a managed Kubernetes service, which has become the de facto technology for CaaS offerings: Amazon Elastic Kubernetes Service (EKS), Azure Kubernetes Service (AKS), Google Kubernetes Engine (GKE), Alibaba Container Service for Kubernetes (ACK), and DigitalOcean Kubernetes.

2.3 Platform as a Service (PaaS)

In the *Platform as a Service* (PaaS) model, the platform provides infrastructure, tools, and APIs that developers can use to build and deploy applications. For example, as a developer, you can build a Java application, package it as a Java Archive (JAR) file, and then deploy it to a platform working according to the PaaS model. The platform provides the Java runtime and other required middleware, and it can also offer extra services like databases or messaging systems. Examples of PaaS offerings are Cloud Foundry, Heroku, AWS Elastic Beanstalk, Azure App Service, Google App Engine, Alibaba Web App Service, and DigitalOcean App Platform. In the past few years, vendors have been converging on Kubernetes for building a new PaaS experience for developers and operators. Examples of this new generation of services are VMware Tanzu Application Platform and RedHat OpenShift.

2.4 Function as a Service (FaaS)

The *Function as a Service* (FaaS) model relies on serverless computing to let consumers focus on implementing the business logic of their applications (often in the form of functions), whereas the platform takes care of providing servers and the rest of the infrastructure. Serverless applications are triggered by events, such as HTTP requests or messages. For example, you might code a function that analyzes a data set whenever available from a message queue and computes results according to some algorithms. Examples of commercial FaaS offerings are Amazon AWS Lambda, Microsoft Azure Functions, Google Cloud Functions, and Alibaba Functions Compute. Examples of open source FaaS offerings are Knative and Apache OpenWhisk.

2.5 Software as a Service (SaaS)

The service with the highest abstraction is *Software as a Service* (SaaS). In this model, consumers access applications as users, while the cloud provider manages the whole stack of software and infrastructure. Many companies build their applications, use a CaaS or PaaS model to run them, and then sell their usage to the end customers as SaaS. The consumers of SaaS applications typically use thin clients like web browsers or mobile devices to access them. Examples of applications available as SaaS are Proton Mail, GitHub, Plausible Analytics, and Microsoft Office 365.

Platform vs. PaaS

Platform is a term that might generate some confusion in a cloud native discussion, so let's clarify. In general, a platform is an operating environment you use to run and manage your applications. Google Kubernetes Engine (GKE) is a platform that offers cloud services according to the CaaS model. Microsoft Azure Functions is a platform that provides cloud services following the FaaS model. At a lower level, if you deploy your applications directly on an Ubuntu machine, that will be your platform. In the rest of the book, whenever I use the term *platform*, I mean the broader concept just explained, unless specified otherwise.

1.3 Properties of cloud native applications

The scene is set: you are in the cloud. How should you design applications to take advantage of its characteristics?

The CNCF identifies five main properties that cloud native applications should have: scalability, loose coupling, resilience, observability, and manageability. Cloud native is a methodology for building and running applications that exhibit those properties. Cornelia Davis sums it up by stating that "cloud-native software is defined by how you compute, not about where you compute."[5] In other words, the cloud is about *where*, and cloud native is about *how*.

I have already covered the *where* part: the cloud. Let's go ahead and explore the *how*. For a quick reference, figure 1.4 lists the properties with short descriptions.

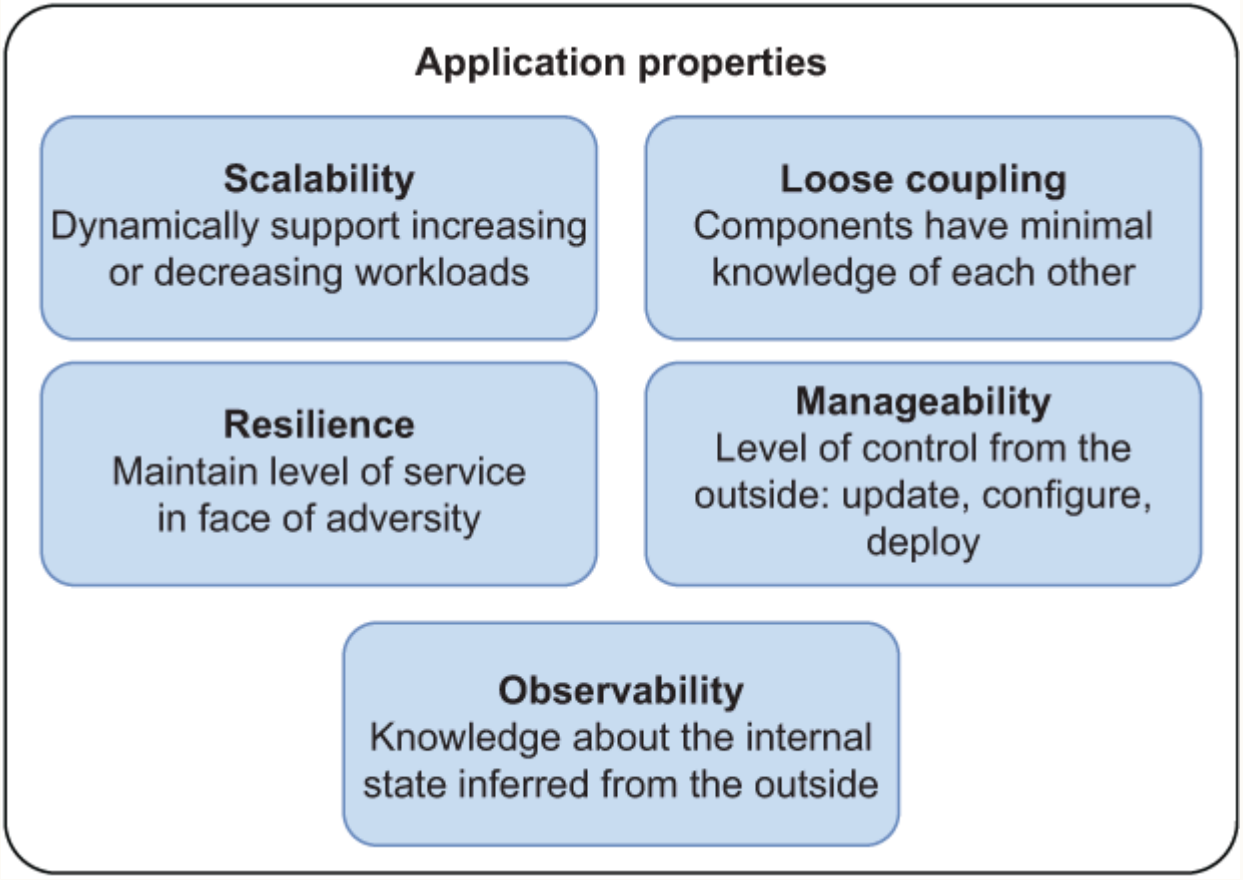


Figure 1.4 The main properties of cloud native applications

3.1 Scalability

Cloud native applications are designed to *scale*, meaning that they can support increasing workloads if provided with additional resources. Depending on the nature of those extra resources, we can distinguish between vertical scalability and horizontal scalability:

- *Vertical scalability* —Scaling vertically, or scaling up or down, means adding hardware resources to or removing them from the computing node, such as CPU or memory. This approach is limited, since it's not possible to keep adding hardware resources. On the other hand, applications don't need to be explicitly designed to be scaled up or down.
- *Horizontal scalability* —Scaling horizontally, or scaling out or in, means adding more computing nodes or containers to, or removing them from, the system. This approach doesn't have the same limits as vertical scalability, but it requires applications to be scalable.

Traditional systems would usually adopt vertical scalability in the event of increasing workloads. Adding CPU and memory was a common approach to enable an application to support more users without redesigning it for scalability. This is still a good option in specific scenarios, but we need something else for the cloud.

In the cloud, where everything is dynamic and in constant change, horizontal scalability is preferred. Thanks to the abstraction levels offered by the cloud computing models, it's straightforward to spin up new instances of your application rather than increasing the computational power of the machines already running. Since the cloud is elastic, we can scale application instances in and out quickly and dynamically. I discussed elasticity as one of the main characteristics of the cloud: computing resources can be provisioned and released proactively, depending on the demand. Scalability is a prerequisite for elasticity.

Figure 1.5 shows the difference between vertical and horizontal scalability. In the first case, we scale by adding more resources to the existing virtual machine. In the second case, we add another virtual machine to help the existing one process the extra workload.

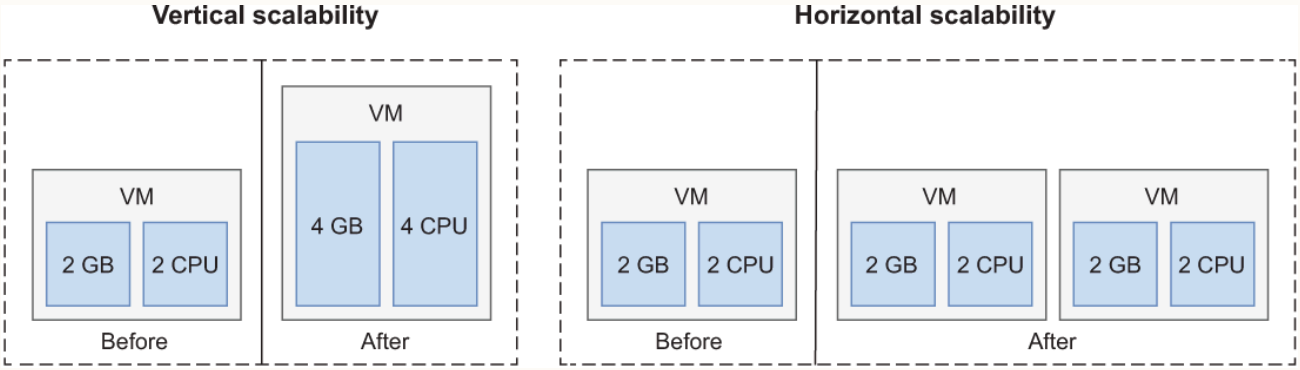


Figure 1.5 When you need to support increasing workloads, the vertical scalability model will add hardware resources to the computing node, while the horizontal scalability model will add more computing nodes.

As you'll see when we discuss Kubernetes, the platform (whether CaaS, PaaS, or something else) takes care of scaling applications in and out dynamically. As developers, it's our responsibility to design applications that can be scaled. The big obstacle to scalability is application state, which essentially is a matter of whether an application is stateful or stateless. Throughout the book, I'll cover techniques for building stateless applications and enabling them to scale without issues. Among other things, I'll show you how to push the application state from Spring to data stores like PostgreSQL and Redis.

3.2 Loose coupling

Loose coupling is an essential property of a system where parts have as little knowledge of each other as possible. The goal is to evolve each piece independently so that when one is changed, the others don't need to change accordingly.

Coupling, and its twin concept *cohesion*, have played an essential role in software engineering for decades. It's a good design practice to decompose a system into modules (*modularization*), each of which has minimal dependencies on the other parts (loose coupling) and to encapsulate code that changes together (high cohesion). Depending on the architectural style, a module can model a monolithic component or a standalone service (for example, a microservice). Either way, we should aim at achieving proper modularization with loose coupling and high cohesion.

Parnas identified three benefits of modularization:[6]

- *Managerial*—Since each module is loosely coupled, the team responsible for it should not need to spend much time coordinating and communicating with other teams.
- *Product flexibility*—The overall system should be flexible since each module is evolved independently of the others.
- *Comprehensibility*—People should be able to understand and work with a module without having to study the whole system.

The preceding benefits are usually among those associated with microservices, but the truth is that you don't need microservices to achieve them. In the last few years, many organizations have decided to migrate from monoliths to microservices. Some of them have failed because they lacked proper modularization. A monolith made up of tightly coupled, non-cohesive components, when migrated, produces a tightly coupled, non-cohesive microservice system, which sometimes is referred to as a *distributed monolith*. I don't consider this a good name because it implies that monoliths are made up of tightly coupled, non-cohesive components by definition. That's not true. The architectural style doesn't matter: a bad design is a bad design. Indeed, I like the *modular monolith* term proposed by Simon Brown to increase awareness that monoliths can promote loose coupling and high cohesion, and that both monoliths and microservices can end up being "big balls of mud."

Throughout the book, I'll discuss some techniques for enforcing loose coupling in applications. In particular, we'll adopt a service-based architecture and focus on building services with clear interfaces to communicate with each other, minimal dependencies on other services, and high cohesion.

3.3 Resilience

A system is resilient if it provides its services even in the presence of faults or environmental changes. Resilience is "the capability of a hardware-software network to provide and maintain an acceptable level of service in the face of faults and challenges to normal operation."[7]

When building cloud native systems, our goal should be to guarantee that our applications are always available, whether there is a failure in the infrastructure or in our software. Cloud native applications run in a dynamic environment where everything keeps changing, and faults can and will occur. This cannot be prevented. In the past, we used to consider changes and faults as exceptions, but for highly distributed systems like cloud native ones, changes are not exceptions: they are the rule.

When discussing resilience, it's worth defining three essential concepts: fault, error, and failure:[8]

- *Fault*—A fault is a defect that produces an incorrect internal state either in the software or the infrastructure. For example, a method call returns a null value, even if its specification mandates that a non-null value is returned.
- *Error*—An error is a discrepancy between the expected behavior of a system and the actual one. For example, due to the preceding fault, a `NullPointerException` is thrown.
- *Failure*—When a fault is triggered and results in an error, a failure might occur, making the system unresponsive and unable to behave according to its specifications. For example, if the `NullPointerException` is not caught, the error provokes a failure: the system responds to any request with a 500 response.

Faults can become errors, which may provoke failures, so we should design applications to be *fault tolerant*. An essential part of resilience is ensuring that a failure will not cascade to other components of the system but stay isolated while it gets fixed. We also want the system to be *self-repairing* or *self-healing*, and the cloud model can enable that.

Throughout the book, I'll show you some techniques for tolerating faults and preventing their effects from propagating to other parts of the system and spreading the failure. For example, we'll use patterns like circuit breakers, retries, timeouts, and rate limiters.

3.4 Observability

Observability is a property that comes from the world of control theory. If you consider a system, observability is a measure of how well you can infer its internal state from its external outputs. In the software engineering context, the system is a single application or a distributed system as a whole. The external outputs can be data like metrics, logs, and traces. Figure 1.6 shows how observability works.

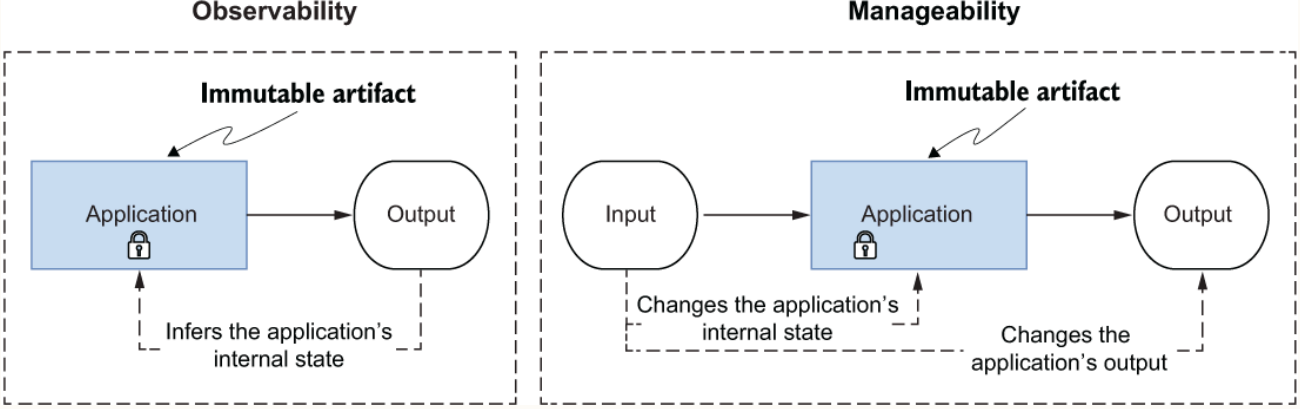


Figure 1.6 Observability is about inferring the internal state of an application from its external outputs. Manageability is about changing the internal state and outputs via external inputs. In both cases, the application artifact is never changed. It's immutable.

The Observability Engineering team at Twitter identifies four pillars of observability:[9]

- *Monitoring*—Monitoring is about measuring specific aspects of an application to get information on its overall health and identify failures. In this book, you'll learn about the useful monitoring features of Spring Boot Actuator and integrate Prometheus with Spring to export relevant metrics about an application.
- *Alerting/visualization*—Collecting data about the state of a system is useful only if it's used to take some action. When a failure is identified while monitoring an application, an alert should be triggered, and some action should be taken to handle it. Specific dashboards are used to visualize the data collected and plot them in relevant graphs to provide a good picture of the system's behavior. This book will show how to use Grafana to visualize data collected from cloud native applications.
- *Distributed systems tracing infrastructure*—In a distributed system, it's not enough to track every single subsystem's behavior. It's essential to trace the data flowing through the different subsystems. In this book, you'll integrate Spring with OpenTelemetry and use Grafana Tempo to collect and visualize the traces.
- *Log aggregation/analytics*—Keeping track of the main events in an application is critical to infer the software's behavior and debug it if something goes wrong. In a cloud native system, logs should be aggregated and collected to provide a better picture of the system's behavior and ensure the possibility of running analytics to mine information from that data. Throughout the book, I'll talk more about logs. You'll use Fluent Bit, Loki, and Grafana to collect and visualize logs, and you'll learn the best practices for logging in a cloud native context.

3.5 Manageability

In control theory, the counterpart of observability is controllability—the ability of external input to change the state or the output of a system in a finite time interval. This con-

cept leads us to the last of the main properties of cloud native: manageability.

Drawing from control theory again, we can say that manageability measures how easily and efficiently an external input can change the state or the output of a system. In less mathematical terms, it's the ability to modify an application's behavior without needing to change its code. This is not to be confused with *maintainability*, which measures how easily and efficiently you can change a system from the inside by changing its code. Figure 1.6 shows how manageability works.

One aspect of manageability is deploying and updating applications while keeping the overall system up and running. Another element is configuration, which I'll address in depth throughout the book. We want to make cloud native applications configurable so we can modify their behavior without changing their code and building a new release. It's common to make configurable settings like data source URLs, service credentials, and certificates. For example, depending on the environment, you may use different data sources: one for development, one for testing, and one for production. Other types of configuration could be feature flags, which determine whether specific features should be enabled at runtime. I'll show you different strategies for configuring applications throughout the book, including using Spring Cloud Config Server, Kubernetes ConfigMaps and Secrets, and Kustomize.

Manageability is not only about particular changes themselves, but also about how easily and efficiently you can apply those changes. Cloud native systems are complex, so it's essential to design applications that can adapt to changing requirements regarding functionality, the environment, and security. Due to this complexity, we should aim at managing as much as possible through automation, which leads us to the last of the Three Ps of Cloud Native: practices.

1.4 Culture and practices supporting cloud native

This section will focus on the last sentence of the definition of cloud native technologies provided by the CNCF: "Combined with robust automation, they allow engineers to make high-impact changes frequently and predictably with minimal toil." I'll discuss three concepts: automation, continuous delivery, and DevOps (figure 1.7).

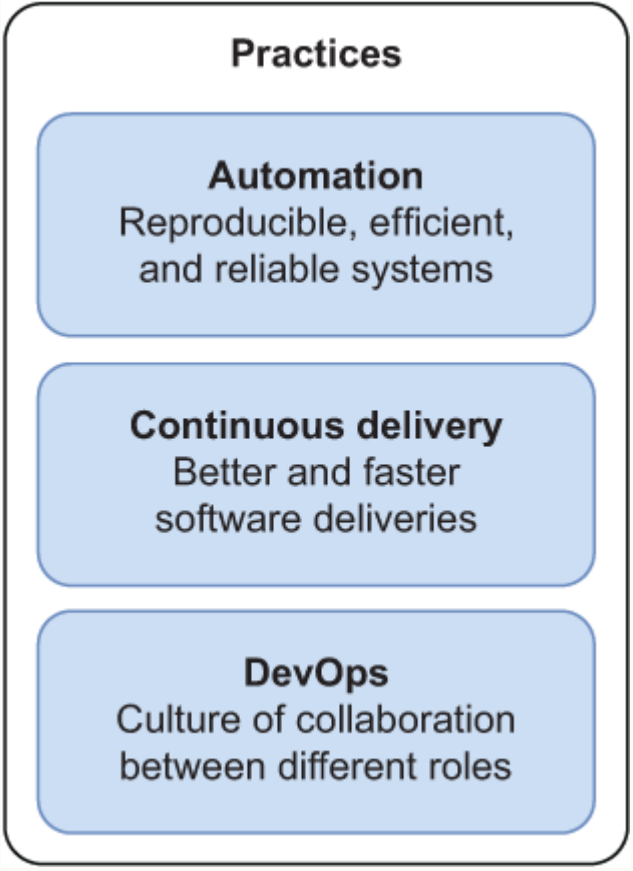


Figure 1.7 Culture and practices for cloud native development

4.1 Automation

Automation is a core tenet of cloud native. The idea is to automate repetitive manual tasks to accelerate the delivery and deployment of cloud native applications. Many tasks can be automated, from building applications to deploying them, from provisioning infrastructure to managing configuration. The most important advantage of automation is that it makes processes and tasks repeatable and overall systems more stable and reliable. Manually executing a task is error-prone and costs money. By automating it, we can get a result that is both more reliable and more efficient.

In the cloud computing model, computing resources are provisioned in an automated, self-service model, and they can be increased or decreased elastically. Two significant categories of automation for the cloud are infrastructure provisioning and configuration management. We call them *infrastructure as code* and *configuration as code*.

Martin Fowler defines *infrastructure as code* as "the approach to defining computing and network infrastructure through source code that can then be treated just like any software system."[10]

Cloud providers offer convenient APIs for creating and provisioning servers, networks, and storage. By automating those tasks with a tool like Terraform, putting the code in source control, and applying the same test and delivery practices used for application

development, we get a more reliable and predictable infrastructure, which is reproducible, more efficient, and less risky. A simple example of an automated task could be creating a new virtual machine with 8 CPUs, 64 GB of memory, and Ubuntu 22.04 as the operating system.

After we've provisioned computing resources, we can manage them and automate their configuration. Paraphrasing the previous definition, *configuration as code* is the approach to defining the configuration of computing resources through source code, which can be treated just like any software system.

Using tools like Ansible, we can specify how a server or a network should be configured. For example, after provisioning the Ubuntu server from the previous paragraph, we can automate the task of installing the Java Runtime Environment (JRE) 17 and opening the ports 8080 and 8443 from the firewall. Configuration as code applies to application configuration as well.

By automating all the infrastructure provisioning and configuration management tasks, we can avoid unstable, unreliable *snowflake servers*. When each server is provisioned, managed, and configured manually, the result is a *snowflake* : a fragile, unique server that cannot be reproduced and is risky to change. Automation helps avoid snowflakes in favor of *phoenix servers*: all tasks acting on those servers are automated, every change can be tracked in source control, reducing risks, and each setup is reproducible. By taking this concept to its extreme, we achieve the so-called *immutable servers*, also mentioned by the CNCF in its cloud native definition as immutable infrastructure.

NOTE You might have heard the expression "pets vs. cattle" when comparing traditional snowflake infrastructure (requiring a lot of care and attention, like pets) and immutable infrastructure or containers (characterized by being disposable and replaceable, like cattle). I won't use this expression in the book, but it's sometimes used in discussions about this subject, so you should be aware of it.

After their initial provisioning and configuration, immutable servers are not changed: they are immutable. If any change is necessary, it's defined as code and delivered. A new server is then provisioned and configured from the new code while the previous server is destroyed.

For example, you have two options if your current infrastructure consists of Ubuntu 20.04 servers and you want to upgrade to Ubuntu 22.04. The first option is to define the upgrade via code and run the automation scripts to perform the operation on the existing machines (phoenix servers). The second option is to automate the provisioning of

new machines running Ubuntu 22.04 and start using those (immutable servers), rather than performing the upgrade on existing machines.

In the next section, I'll talk about automation for building and deploying applications.

4.2 Continuous delivery

Continuous delivery is "a software development discipline where you build software in such a way that the software can be released to production at any time."[11] With continuous delivery, teams implement features in short cycles, ensuring that the software can be released at any time reliably. Such a discipline is key to "make high-impact changes frequently and predictably with minimal toil," as per the cloud native definition from the CNCF.

Continuous integration (CI) is a foundational practice in continuous delivery. Developers commit their changes to the mainline (the main branch) continuously (at least once a day). At each commit, the software is automatically compiled, tested, and packaged as executable artifacts (such as JAR files or container images). The idea is to get fast feedback about the software's status after each new change. If an error is detected, it should be immediately fixed to ensure the mainline keeps being a stable foundation for further development.

Continuous delivery (CD) builds on CI and focuses on keeping the mainline always healthy and in a releasable state. After an executable artifact is produced as part of the integration with the mainline, the software is deployed to a production-like environment. It goes through additional tests to assess its *releasability*, such as user acceptance tests, performance tests, security tests, compliance tests, and any other tests that might increase the confidence that the software can be released. If the mainline is always in a releasable state, releasing a new version of the software becomes a business decision instead of a technical one.

Continuous delivery encourages the automation of the whole process via a *deployment pipeline* (also called a *continuous delivery pipeline*), as described in the foundational book *Continuous Delivery* by Jez Humble and David Farley (Addison-Wesley Professional, 2010). A deployment pipeline goes from code commit to a releasable outcome, and it's the only way to production. Throughout the book, we'll build a deployment pipeline to keep the main branch of our applications always in a releasable state. In the end, we'll use it to deploy applications automatically to a Kubernetes cluster in production.

Sometimes continuous delivery is confused with *continuous deployment*. The former approach makes sure that after every change, the software is in a state in which it can be

deployed to production. When that's actually done is a business decision. With continuous deployment we add one last step to the deployment pipeline to automatically deploy a new release in production after every change.

Continuous delivery is not about tools. It's a discipline that involves cultural and structural changes in your organization. Setting up an automated pipeline to test and deliver your applications doesn't mean that you're doing continuous delivery. Similarly, using a CI server to automate builds doesn't mean you're doing continuous integration.[12] That leads us to the next topic, which is also commonly mistaken as being about tools.

Continuous delivery vs. CI/CD

Since continuous integration is a foundational practice of continuous delivery, the combination is often referred to as CI/CD. As a consequence, deployment pipelines are often called CI/CD pipelines. I have some reservations about the term, because continuous integration is not the only practice included in the continuous delivery discipline. For example, test-driven development (TDD), automated configuration management, acceptance testing, and continuous learning are equally important.

The *CI/CD* term is not used by Jez Humble and Dave Farley in their *Continuous Delivery* book, nor in any other book they have written on the subject. Also, it can be confusing. Does *CD* stand for *continuous delivery* or *continuous deployment*? In this book, I will refer to the holistic approach of delivering "better software faster"[a] as *continuous delivery* rather than CI/CD.

[a] D. Farley, *Continuous Delivery Pipelines*, 2021.

4.3 DevOps

DevOps is another of those buzzwords that are pretty popular these days but too often misunderstood. When shifting to cloud native, DevOps is an important concept to grasp.

The origins of DevOps are peculiar. One of the curious aspects is that the creators of this concept didn't initially provide a definition. The result was that several people used their own interpretations and, of course, we ended up using DevOps to mean different things.

NOTE If you're interested in knowing more about the origins of DevOps, I suggest watching the Ken Mugrage talks available on YouTube: "DevOps and DevOpsDays—Where it started, where it is, where it's going" (**http://mng.bz/Ooln**).

Among all the definitions of DevOps, I find the one proposed by Ken Mugrage (principal technologist at ThoughtWorks) to be particularly informative and interesting. He highlights what I believe is the true meaning of DevOps.

A culture where people, regardless of title or background, work together to imagine, develop, deploy, and operate a system.[13]

So DevOps is a culture, and it's all about working together toward a common goal. Developers, testers, operators, security experts, and other people, regardless of title or background, work together to bring ideas to production and produce value.

It means the end of *silos*—no more walls between feature teams, QA teams, and operations teams. DevOps is often considered a natural continuation of agile, which is an enabler for DevOps with the concept of small teams delivering value to customers frequently. A concise way to describe DevOps is with a famous sentence that Werner Vogels, Amazon CTO, pronounced in 2006 when DevOps was not even a thing yet: "You build it, you run it."[14]

Having defined what DevOps is, I'll briefly mention what it isn't:

- *DevOps doesn't mean NoOps.* It's a common mistake to think that developers take care of operations and that the operator's role disappears. Instead, it's a collaboration. A team will include both roles, contributing to the overall team's skills required to bring a product from the original idea to production.
- *DevOps is not a tool.* Tools like Docker, Ansible, Kubernetes, and Prometheus are usually referred to as DevOps tools, but that's wrong. DevOps is a culture. You don't turn into a DevOps organization by using particular tools. In other words, DevOps is not a product, but tools are relevant enablers.
- *DevOps is not automation.* Even if automation is an essential part of DevOps, automation is not its definition. DevOps is about developers and operators working together from the original idea to production while possibly automating some of their processes, such as continuous delivery.
- *DevOps is not a role.* If we consider DevOps to be a culture or a mindset, it's hard to make sense of a DevOps role. And yet, there is an increasing request for DevOps engineers. Usually when recruiters search for DevOps engineers, they are looking for skills like proficiency with automation tools, scripting, and IT systems.

- *DevOps is not a team.* Organizations not fully understanding the preceding points risk keeping the same silos as before, with one change: replacing the Ops silo with a DevOps silo, or, even worse, simply adding a new DevOps silo.

The collaboration between developers and operators is paramount when going cloud native. As you may have noticed, designing and building cloud native applications requires that you always keep in mind where you're going to deploy those applications: the cloud. Working together with operators allows developers to design and build higher-quality products.

It's called DevOps, but remember that the definition doesn't apply only to developers and operators. Instead, it generically refers to people, regardless of title or background. It means that collaboration also involves other roles like testers and security experts (though we probably don't need new terms like DevSecOps, DevTestOps, DevSecTestOps, or DevBizSecTestOps). Together they are all responsible for the entire product life cycle and are key to achieving the goals of continuous delivery.

1.5 Is the cloud your best option?

One of the biggest mistakes in our industry is deciding to adopt a technology or approach just because it's new and everyone is talking about it. There are endless stories about companies migrating their monoliths to microservices and ending up with disastrous failures. I have already explained the properties of the cloud and of cloud native applications. Those should provide you with some guidance. If your system doesn't need those properties because it doesn't have the problems they are trying to solve, chances are that "going cloud native" is not the best option for your project.

As technologists, it's easy for us to get caught up in the latest, trendiest, shiniest technology. The point is to figure out whether a specific technology or approach can solve *your* problems. We turn ideas into software that we deliver to our customers and that provides them with some value. That's our end goal. If a technology or approach helps you provide more value to your customers, you should consider it. If it's not worthy, and you decide to go with it anyway, you'll likely end up having higher costs and many problems.

When is moving to the cloud a good idea? Why are companies adopting the cloud native approach? The main reasons for going cloud native, illustrated in figure 1.8, are speed, scale, resilience, and cost. If your business vision includes those goals and faces the same problems that cloud technologies try to solve, then it's good to consider moving to the cloud and adopting a cloud native approach. Otherwise, it might be better to stay on

the ground. For example, if your company is providing services through a monolithic application in its maintenance phase that will not be further expanded with new features, and that has performed well in the last decade, there might be no good reason to migrate it to the cloud, let alone to turn it into a cloud native application.

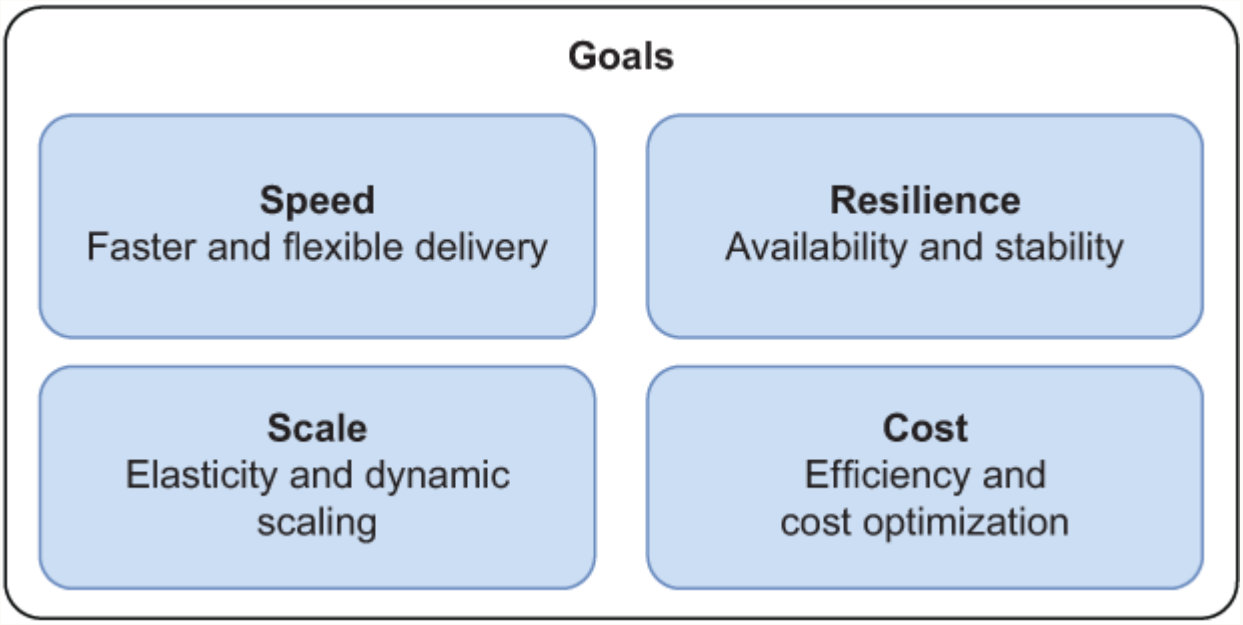


Figure 1.8 Going cloud native can help you meet several goals related to speed, resilience, scale, and cost optimization.

5.1 Speed

Being able to deliver software faster is an important goal for enterprises nowadays. Bringing ideas into production as quickly as possible, therefore reducing the time to market, is a critical competitive advantage. Going to production with the right idea at the right time might make the difference between success and failure.

Customers expect to have more and more features implemented or bugs fixed, and they want them now. They will not be happy to wait six months for the next release of your software. Their expectations keep increasing, and you need a way to keep up with them. In the end, it's all about providing value to customers and ensuring they are satisfied with the results. Otherwise, your business will not survive the fierce competition out there.

Delivering faster and frequently is not only about competition and customer deadlines. It's also about making the feedback cycle shorter. Frequent and small releases mean that you can get feedback from customers sooner. A shorter feedback loop, in turn, decreases the risk associated with the new functionality you're releasing. Instead of spending months trying to implement the perfect feature, you can get it out there sooner, get

feedback from customers, and adjust it to align with their expectations. Also, smaller releases contain fewer changes, thus reducing the number of parts that could fail.

Flexibility is also needed, since customers expect the continuous evolution of your software. For example, it should be flexible enough to support new types of clients. Nowadays, more and more objects in our daily lives have been connected to the internet, such as all kinds of mobile and IoT systems. You'll want to be open to any future extensions and client types so you can provide business services in new ways.

The traditional software development approach does not support this goal. Too often it is characterized by massive releases, little flexibility, and extended release cycles. The cloud native approach, combined with automated tasks, continuous delivery workflows, and DevOps practices, helps businesses go faster and reduce the time to market.

5.2 Resilience

Everything changes, and failures happen all the time. Gone are the times when we tried to predict failures and treat them as exceptions. As I mentioned before, changes are not exceptions. They are the rule.

Customers want software to be available 24/7 and to be upgraded as soon as a new feature is released. Downtime or failures can cause a direct loss of money and customer dissatisfaction. They may even affect one's reputation, damaging the organization's future market opportunities.

Whether there is a failure in the infrastructure or the software, your goal is to guarantee the availability and reliability of your systems, even if only in a degraded operational mode. To guarantee availability, you need to have something in place to face the failures when they arrive, handle them, and ensure that the overall system can still provide services to its users. Any action required to process the failure or tasks like upgrades should require zero downtime. Customers expect that.

We want cloud native applications to be resilient, and cloud technologies provide strategies for implementing resilient infrastructure. If being always available, safe, and resilient is a requirement for your business, the cloud native approach is a good option for you. The resilience of a software system, in turn, enables speed: the more stable the system, the more frequently you can safely release new features.

5.3 Scale

Elasticity is about being able to scale your software depending on the load. You can scale an elastic system to ensure an adequate service level for all your customers. If there is a higher load than usual, you'll need to spin up more instances of your services to support that extra traffic. Or perhaps something terrible happened and some services failed—you'll need to be able to spin up new instances to replace them.

Foreseeing what will happen is hard, if not impossible. It's not enough to build scalable applications—you need them to scale dynamically. Whenever there is a high load, your system should scale out dynamically, quickly, and painlessly. When the high peak is over, it should scale in again.

If your business needs to adapt quickly and efficiently to new customers or requires the flexibility to support new types of clients (which increase the workload on the servers), the cloud's very nature can provide you with all the elasticity you need, in combination with cloud native applications that are scalable by definition.

5.4 Cost

As a software developer, you might not deal directly with money, but it's your responsibility to take cost into account when designing a solution. The cloud computing model helps optimize IT infrastructure costs with its elasticity and on-demand pay-per-use policies. No more always-on infrastructure: you provision resources when you need them, pay for the actual usage, and then destroy them when you don't need them anymore.

On top of that, adopting the cloud native approach leads to further cost optimizations. Cloud native applications are designed to be scalable so they can take advantage of the cloud's elasticity. They are resilient, so the costs related to downtime and hard failures in production are lower. Being loosely coupled, they enable teams to go faster and speed up the time to market, with notable competitive advantages. And the list goes on.

The hidden costs of moving to the cloud

Before deciding to migrate to the cloud, it's essential also to consider other types of costs. On the one hand, you can optimize costs by paying for only what you use. But on the other hand, you should consider the cost of migrating and its consequences.

Migrating to the cloud requires specific competencies that employees might not have yet. This may mean investing in their education to acquire the necessary skills and perhaps hiring professionals as consultants to help with the migration to the cloud. Depending on the chosen solution, the organization may need to take over some extra responsibilities, such as handling security in the cloud, which in turn requires specific skills. There are also other considerations, like business interruptions during the migration, retraining end users, and updating documentation and support materials.

1.6 Cloud native topologies

My explanation of cloud native didn't involve specific technologies or architectures. The CNCF mentions some in its definition, like containers and microservices, but those are just examples. You don't have to use Docker containers for your applications to be cloud native. Think about serverless or PaaS solutions. Writing functions for the AWS Lambda platform or deploying applications to Heroku doesn't require you to build containers. Still, they are categorized as cloud native.

In this section, I'll describe some common cloud native topologies (see figure 1.9). First, I'll introduce the concepts of containers and orchestration, which will be further explored later when I discuss Docker and Kubernetes. Then I'll present the topic of serverless technologies and functions (FaaS). I will not focus much on the FaaS model in this book, but I will cover the basics of how to build serverless applications using Spring Native and Spring Cloud Function.

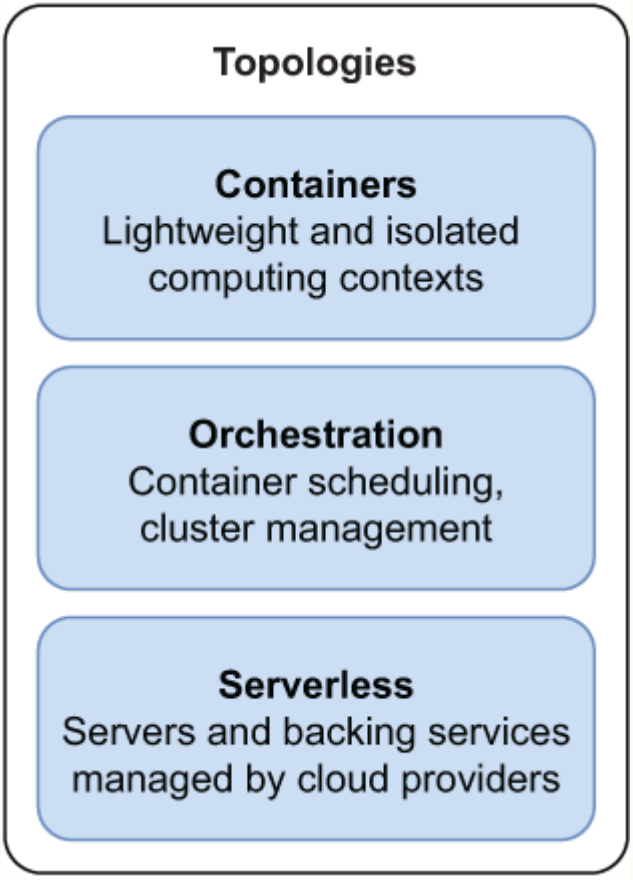


Figure 1.9 The main cloud native computing models are containers (managed by orchestrators) and serverless.

6.1 Containers

Imagine you join a team and start working on an application. The first thing you do is follow a guide to set up your local development environment that's similar to the ones used by your colleagues. You develop a new feature and then test it in a quality assurance (QA) environment. Once it's verified, the application can be deployed to staging for some extra testing and finally to production. The application is built to run in an environment with specific characteristics, so it's essential to have all the different environments as similar as possible. How would you do that? That's where containers enter the scene.

Before containers, you would rely on virtual machines to guarantee the reproducibility of the environment, isolation, and configurability. Virtualization works by leveraging a hypervisor component that abstracts the hardware, making it possible to run multiple operating systems on the same machine in an isolated fashion. The hypervisor would run directly on the machine hardware (type 1) or on the host operating system (type 2).

On the other hand, an *OS container* is a lightweight executable package that includes everything needed to run the application. Containers share the same kernel as the host: there's no need to bootstrap full operating systems to add new isolated contexts. On Linux, that is possible by leveraging a couple of features offered by the Linux kernel:

- *namespaces* for partitioning resources among processes so that each process (or group of processes) can only see a subset of the resources available on the machine
- *cgroups* for controlling and limiting the resource usage for a process (or group of processes)

NOTE When using virtualization only, the hardware is shared, whereas containers also share the same operating system kernel. Both provide computing environments for running software in isolation, even if the degree of isolation is not the same.

Figure 1.10 shows the difference between virtualization and container technologies.

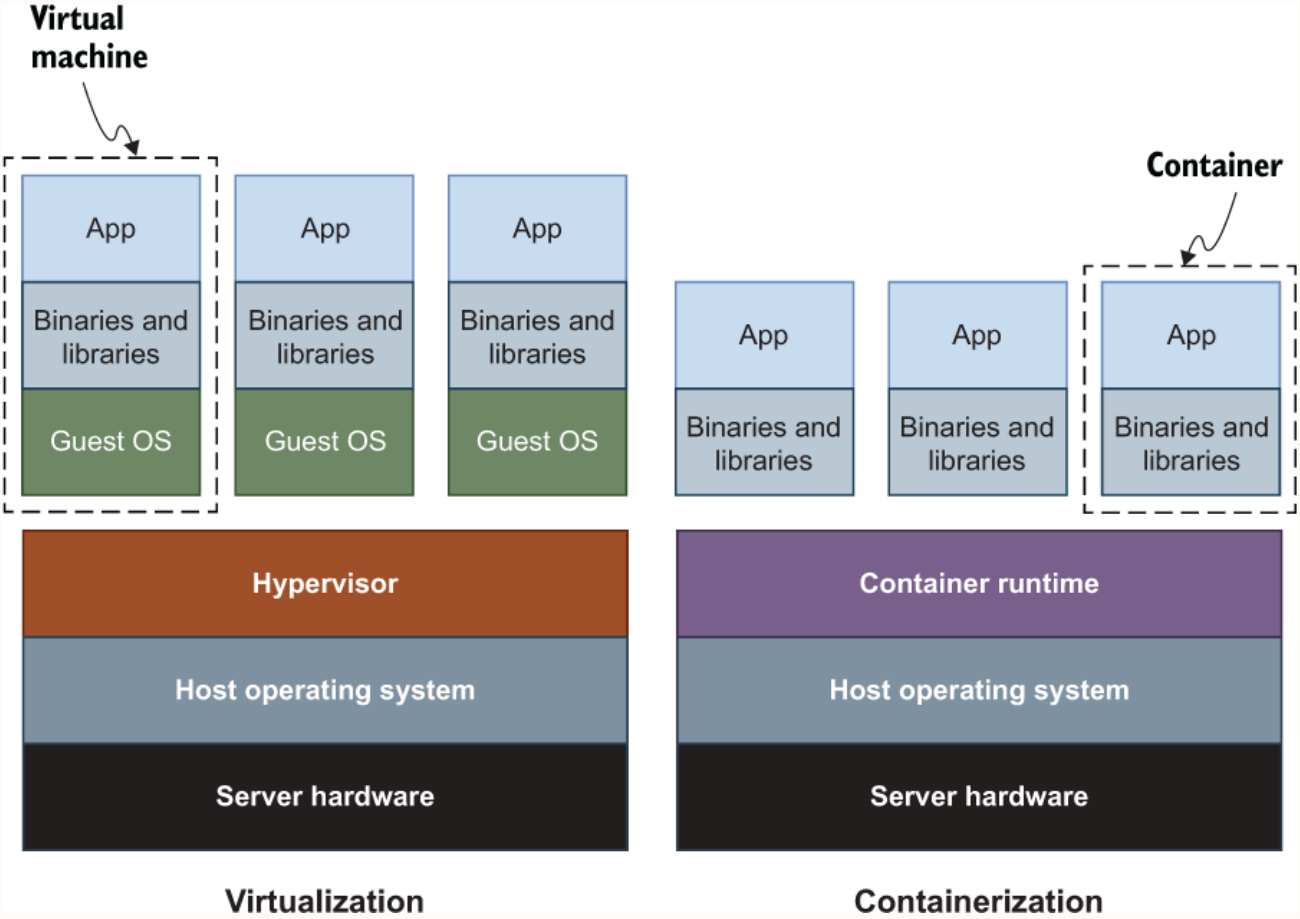


Figure 1.10 Virtualization and container technologies differ in what is shared across isolated contexts. Virtual machines share the hardware only. Containers share the operating system kernel as well. Containers are more lightweight and portable.

Why are containers so popular for cloud native applications? Traditionally, you would have to install and maintain the Java Runtime Environment (JRE) and middleware on a virtual machine to make your application run. Instead, a container can run reliably in almost any computing environment, independent of the application, its dependencies, or middleware. It doesn't matter which kind of application it is, in which language it's written, or which libraries it uses. All containers have a similar shape from the outside, just like the containers used for shipping.

Therefore, containers enable agility, portability across different environments, and deployment repeatability. Being lightweight and less resource-demanding, they are perfect for running in the cloud, where applications are disposable and scaled dynamically and quickly. In comparison, building and destroying virtual machines is much more expensive and time-consuming.

Containers! Containers everywhere!

Container is one of those words that can mean different things. Sometimes this ambiguity can generate confusion, so let's see what it means in specific contexts.

- *OS*—An OS container is a method for running one or more processes in an environment isolated from the rest of the system. This book will focus on *Linux containers*, but note that Windows containers also exist.
- *Docker*—A Docker container is an implementation of a Linux container.
- *OCI*—An OCI container is a standardization of the Docker container implementation by the Open Container Initiative (OCI).
- *Spring*—A Spring container is the application context where objects, properties, and other application resources are managed and executed.
- *Servlet*—A Servlet container provides a runtime for web applications leveraging the Java Servlet API. The Catalina component from the Tomcat server is an example of a Servlet container.

Virtualization and containers are not mutually exclusive. You can use them both in a cloud native context, having an infrastructure made up of virtual machines on which you run containers. The Infrastructure as a Service (IaaS) model provides a virtualization layer that you can use to bootstrap new virtual machines. On top of that, you can install a container runtime and run your containers.

An application is usually made up of different containers that can be run on the same machine during development or while performing early testing. But you would quickly reach the point where it gets too complicated to manage many containers, primarily when you start replicating them for scalability and distributing them across different machines. That's when you'll start relying on the higher level of abstraction provided by the Container as a Service (CaaS) model, which offers functionality to deploy and manage containers in clusters of machines. Note that, behind the scenes, there is still a virtualization layer.

Containers are involved even when you're using PaaS platforms like Heroku or Cloud Foundry. You deploy your applications on those platforms by providing just the JAR arti-

fact, since they take care of the JRE, the middleware, the OS, and any needed dependencies. Behind the scenes they build a container out of all those components and finally run it. The difference is that it's no longer you who is responsible for building a container—the platform does it for you. On the one hand, that's convenient, in terms of fewer responsibilities for developers. On the other hand, you are giving up control over the runtime and middleware, and you might face vendor lock-in.

In this book, you'll learn how to use Cloud Native Buildpacks (a CNCF project) to containerize Spring applications, and you'll use Docker to run them in your local environment.

6.2 Orchestration

So you have decided to use containers, great! You can rely on their portability to deploy them to any infrastructure providing a container runtime. You can achieve reproducibility, so there are no bad surprises when moving containers from development to staging to production. You can scale them quickly, since they're so lightweight, and get high availability for your applications. You're ready to adopt them for your next cloud native system. Or are you?

Provisioning and managing containers on a single machine is pretty straightforward. But when you start dealing with tens or hundreds of containers scaled and deployed on several machines, you need something else.

When you move from virtual servers (the IaaS model) to container clusters (the CaaS model), you're also switching your point of view.[15] In IaaS you focus on single computing nodes, which are virtual servers. In CaaS the underlying infrastructure is abstracted, and you're focusing on clusters of nodes.

With the new perspective offered by CaaS solutions, the deployment target will not be a machine anymore but rather a cluster of machines. CaaS platforms, such as those based on Kubernetes, provide many features to address all the significant concerns we look for in a cloud native environment, *orchestrating* containers across machines. The two different topologies are shown in figure 1.11.

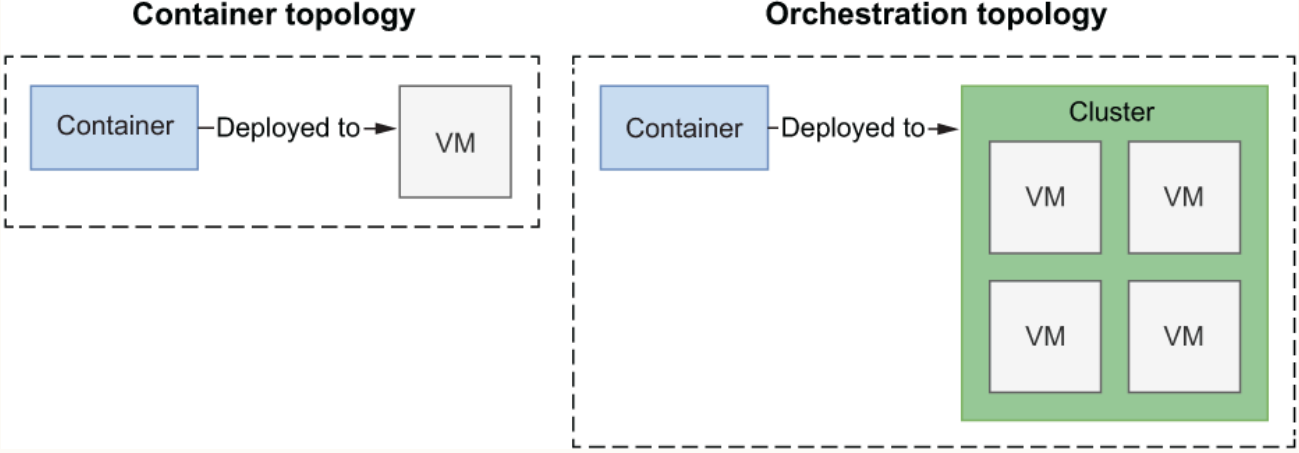


Figure 1.11 The deployment target of containers is a machine, whereas for orchestrators, it's a cluster.

Container orchestration helps you automate many different tasks:

- Managing clusters, bringing up and down machines when necessary
- Scheduling and deploying containers within a cluster to a machine that meets the container requirements for CPU and memory
- Dynamically scaling containers for high availability and resilience, leveraging health monitoring
- Setting up networks for containers to communicate with each other, defining routing, service discovery, and load balancing
- Exposing services to the internet, establishing ports and networks
- Allocating resources to containers according to specific criteria
- Configuring the applications running within the containers
- Ensuring security and enforcing access control policies

Orchestration tools are instructed declaratively, such as through YAML files. Following the format and language defined by the specific tool, you usually describe the state you'd like to achieve; for example, you would like to have three replicas of your web application container deployed in a cluster, exposing its services to the internet.

Examples of container orchestrators are Kubernetes (a CNCF project), Docker Swarm, and Apache Mesos. In this book, you'll learn how to use Kubernetes to orchestrate the containers for your Spring applications.

6.3 Serverless

After moving from virtual machines to containers, we can abstract the infrastructure even more: that's where serverless technologies are placed. The serverless computing

model enables developers to focus on implementing the business logic for their applications.

The name *serverless* might be misleading. Of course there is a server. The difference is that you do not need to manage it or orchestrate the application's deployment on it. That's a platform responsibility now. When you use an orchestrator like Kubernetes, you must still consider infrastructure provisioning, capacity planning, and scaling. In contrast, a serverless platform takes care of setting up the underlying infrastructure needed by the applications, including virtual machines, containers, and dynamic scaling.

Serverless architectures are usually associated with functions, but they comprise two main models that are often used together:

- *Backend as a Service* (BaaS)—In this model, applications rely heavily on third-party services offered by cloud providers, such as databases, authentication services, and message queues. The focus is on reducing development and operational costs related to backend services. Developers can implement frontend applications (such as single-page applications or mobile applications) while offloading most or all of the backend functionality to BaaS vendors. For example, they could use Okta to authenticate users, Google Firebase for persisting data, and Amazon API Gateway to publish and manage REST APIs.
- *Function as a Service* (FaaS)—In this model, applications are stateless, triggered by events, and fully managed by the platform. The focus is on reducing deployment and operations costs related to orchestrating and scaling applications. Developers can implement the business logic for their applications, and the platform takes care of the rest. Serverless applications don't have to be implemented with functions to be categorized as such. There are two main FaaS offerings. One option is to go with vendor-specific FaaS platforms, such as AWS Lambda, Azure Functions, or Google Cloud Functions. Another option is to choose a serverless platform based on open source projects, which can run either in a public cloud or on premises, addressing concerns like vendor lock-in and lack of control. Examples of such projects are Knative and Apache OpenWhisk. Knative provides a serverless runtime environment on top of Kubernetes, as you'll see in chapter 16. It's used as the foundation for enterprise serverless platforms like VMware Tanzu Application Platform, RedHat OpenShift Serverless, and Google Cloud Run.

Serverless applications are typically event-driven and run only when there is an event to handle, such as an HTTP request or a message. The event can be external or be produced by another function. For example, a function might be triggered whenever a message is added to a queue, process it, and then exit the execution.

When there is nothing to process, the serverless platform shuts down all the resources involved with the function, so you can really pay for the actual usage. In the other cloud native topologies like CaaS or PaaS, there is always a server running 24/7. Compared to traditional systems, they offer the advantage of dynamic scalability, reducing the number of resources provisioned at any given time. Still, there is always something up and running, and it has a cost. In the serverless model, resources are provisioned only when necessary. If there is nothing to process, everything is shut down. That's what we call *scaling to zero*, and it's one of the main features offered by serverless platforms.

Besides cost optimization, serverless technologies also move some extra responsibility from the application to the platform. That might be an advantage, since it allows developers to focus exclusively on the business logic. But it's also essential to consider what degree of control you would like to have and how you will deal with vendor lock-in. Each FaaS, and in general serverless, platform has its own features and APIs. Once you start writing functions for a specific platform, you can't move them easily to another, as you would do with containers. With FaaS, you might compromise more than with any other approach—prioritizing responsibility and scope at the cost of control and portability. That's why Knative became popular quickly: it is built on Kubernetes, which means that you can easily move your serverless workloads between platforms and vendors. In the end, it's a matter of tradeoffs.

1.7 Architectures for cloud native applications

We have reached the last step of our journey to define cloud native, and I've introduced the primary characteristics we'll rely on throughout the book. In the previous section, you got familiar with the main cloud native topologies, particularly containers, which are our computation units. Now let's look at what's inside and explore some high-level principles involved in architecting and designing cloud native applications. Figure 1.12 shows the main concepts covered in this section.

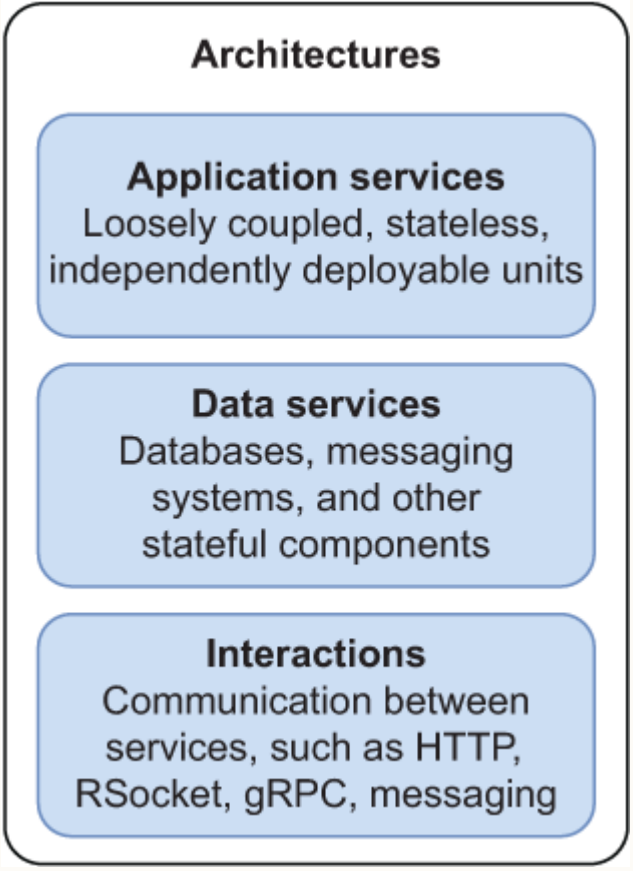


Figure 1.12 Cloud native architectural elements

7.1 From multi-tiered to microservices architectures and beyond

IT infrastructures have always affected the way software applications are architected and designed. In the beginning, monolithic applications were deployed on huge mainframes as single components. When the internet and PCs became popular, we started designing applications according to the client/server paradigm. A multi-tiered architecture, relying on that paradigm, was widely used for desktop and web applications, decomposing the code into presentation, business, and data layers.

With the increase in application complexity and the need for agility, new ways of further decomposing the code have been explored, and a new architectural style entered the stage: microservices. In the last few years, this architectural style has become more and more popular, and many companies decided to refactor their applications according to this new style. Microservices are usually compared to monolithic applications, as shown in figure 1.13.

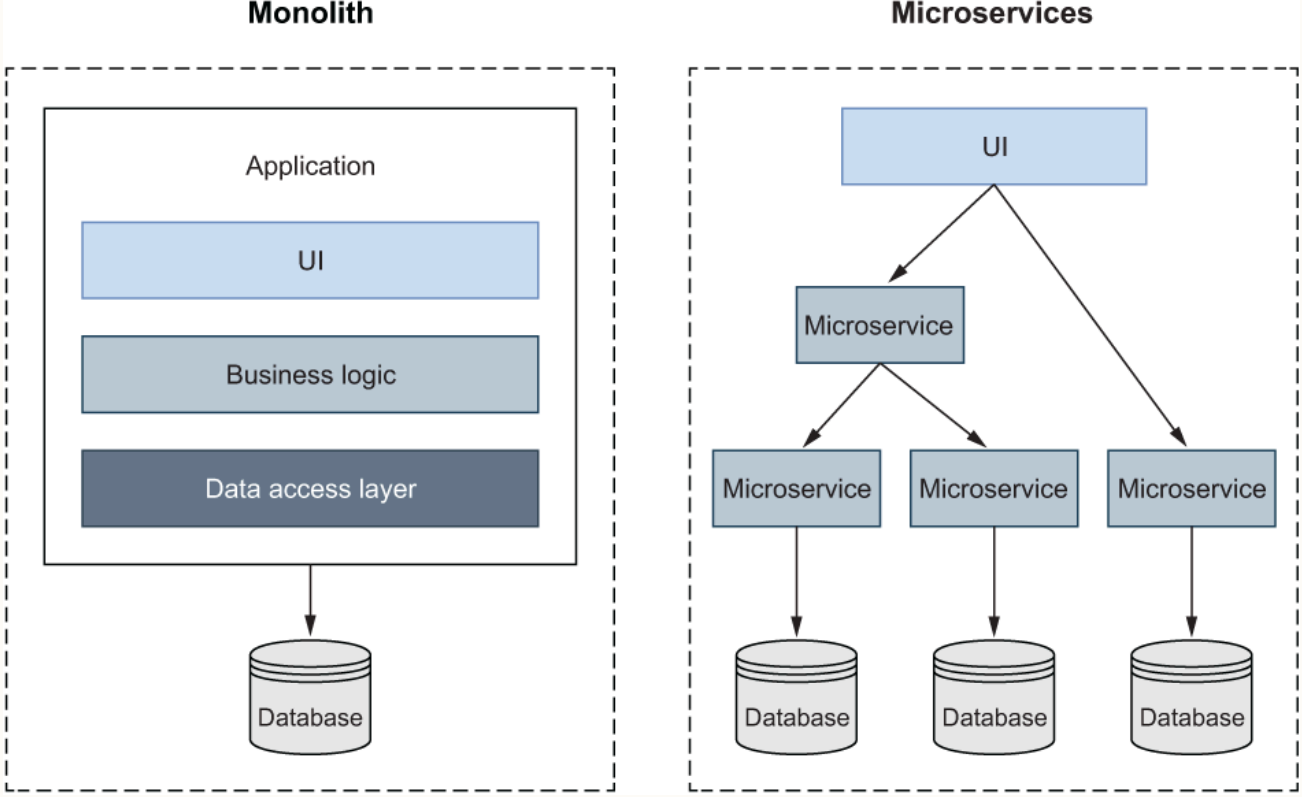


Figure 1.13 Monolithic applications vs. microservices. Monolithic architectures are often multi-tiered. Microservices are made up of different components that can be deployed independently.

The main difference lies in how the application has been decomposed. A monolithic application is associated with using three large layers. A microservice-based application, in contrast, is associated with many components, each implementing only one piece of functionality. Many patterns have been proposed to decompose a monolith into microservices and to handle the complexity created by having many components instead of one.

NOTE This book is not about microservices, so I won't go into detail about them. If you are interested in the topic, you can check out *Building Microservices*, second ed., by Sam Newman (O'Reilly, 2021) and *Microservices Patterns* by Chris Richardson (Manning, 2018). For a more Spring-oriented analysis, you can find *Spring Microservices in Action*, second ed., by John Carnell and Illary Huaylupo Sanchez (Manning, 2021) in the Manning catalog. If you are not familiar with microservices, don't worry. That knowledge is not required to follow along with this book.

After many years of fame and failed migrations, intense discussions about the future of this popular architectural style have arisen in the developer community. Some engineers started talking about *macroservices* to reduce the number of components and, therefore, the complexity of managing them. The term "macroservices" was suggested sardonically by Cindy Sridharan, but it has been adopted in the industry and used by companies like Dropbox and Airbnb to describe their new architectures.[16] Others proposed a *citadel* ar-

chitectural style, consisting of a central monolith surrounded by microservices. Still others are advocating a return to monolithic applications in the form of modular monoliths.

In the end, the important thing is to choose an architecture that can deliver value to our customers and our businesses. That's why we develop applications in the first place. Each architectural style has its use cases. There is no such thing as a silver bullet or one-size-fits-all solution. Most of the negative experiences related to microservices were caused by other issues, such as bad code modularization or unfitting organization structure. There shouldn't be a battle between monoliths and microservices.

In this book, I'm interested in showing you how to build cloud native applications with Spring and deploy them to Kubernetes as containers. Cloud native applications are distributed systems, just like microservices are. Some topics usually discussed in the context of microservices actually belong to any distributed system, such as routing and service discovery. Cloud native applications are loosely coupled by definition, and that is also a feature of microservices.

Even if they have some similar aspects, it's essential to understand that cloud native applications and microservices are not the same. You can definitely use a microservices style for cloud native applications. Many developers do, but it's not a requirement. In this book, I will use an architectural style that we might call *service-based*. Perhaps it's not a catchy name or a fancy one, but it's enough for our purpose. We'll deal with services. They can be of any size, and they can encapsulate logic according to different principles. That doesn't matter. What we want is to design services to fit our development, organizational, and business needs.

7.2 Service-based architecture for cloud native applications

Throughout this book, we'll design and build cloud native applications according to a service-based architecture.

Our central unit of work will be a service that can interact with other services in different ways. Using the distinction proposed by Cornelia Davis in her book *Cloud Native Patterns* (Manning, 2019), we can identify two elements of the architecture: services and interactions.

- *Service*—A component that provides any kind of service to another component
- *Interaction*—The communication of services with each other to accomplish the system's requirements

Services are quite generic components—they might be anything. We can classify them according to whether they store any kind of state, distinguishing between *application services* (stateless) and *data services* (stateful).

Figure 1.14 shows the elements of a cloud native architecture. An application used to manage the inventory for a book library would be an application service. A PostgreSQL database used to store information about the books would be a data service.

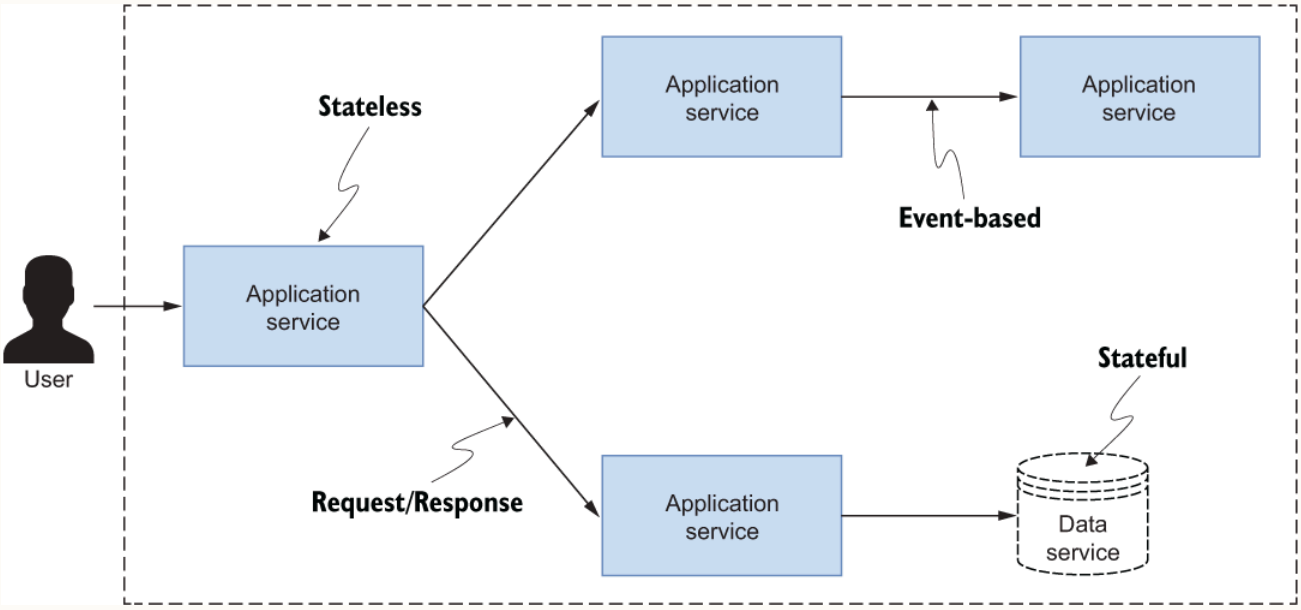


Figure 1.14 Service-based architecture for cloud native applications. The main elements are services (application or data) that interact with each other in different ways.

APPLICATION SERVICES

Application services are stateless and are responsible for implementing any kind of logic. They don't have to obey specific rules like those for microservices, as long as they expose all the cloud native properties you learned about earlier in this chapter.

It's paramount that you design each service with loose coupling and high cohesion in mind. Services should be as independent as possible. Distributed systems are complex, so you should be extra careful during the design phase. Increasing the number of services leads to an increased number of problems.

You'll probably develop and maintain most of the application services in your system by yourself, but you can also use some offered by cloud providers, such as authentication or payment services.

DATA SERVICES

Data services are stateful and are responsible for storing any kind of state. The *state* is everything that should be preserved when shutting down a service or spinning up a new instance.

Data services can be relational databases like PostgreSQL, key/value stores like Redis, or message brokers like RabbitMQ. You can manage these services by yourself. Doing so is more challenging than managing cloud native applications due to the storage required to save the state, but you will gain more control over your own data. Another option is to use the data services offered by a cloud provider, in which case the provider will be responsible for managing all the concerns related to storage, resilience, scalability, and performance. In this case, you can take advantage of the many data services explicitly built for the cloud, such as Amazon DynamoDB, Azure Cosmos DB, and Google BigQuery.

Cloud native data services are a fascinating topic, but we will mainly deal with applications in this book. Data-related concerns like clustering, replication, consistency, and distributed transactions will not be detailed too much in the book. I would love to, but they deserve their own book to be adequately covered.

INTERACTIONS

Cloud native services communicate with each other to fulfill the requirements of the system. How that communication happens will affect the overall properties of the system. For example, choosing a request/response pattern (synchronous HTTP call) over an event-based approach (messages streamed through RabbitMQ) will result in different levels of resilience for the application. In this book, we'll use different types of interactions, learn the differences between them, and see when to use each approach.

Summary

- Cloud native applications are highly distributed systems that are specifically designed for and live in the cloud.
- The cloud is an IT infrastructure provided as a commodity in terms of computing, storage, and networking resources.
- In the cloud, users pay only for the actual resources they use.
- Cloud platforms deliver their services at different levels of abstraction: infrastructure (IaaS), container (CaaS), platform (PaaS), functions (FaaS), or software (SaaS).
- Cloud native applications are horizontally scalable, loosely coupled, highly cohesive, resilient to faults, manageable, and observable.

- Cloud native development is supported by automation, continuous delivery, and DevOps.
- Continuous delivery is a holistic engineering practice for delivering high-quality software quickly, reliably, and safely.
- DevOps is a culture enabling collaboration among different roles to deliver business value together.
- Modern businesses go cloud native to produce software that can be delivered quickly, can be scaled dynamically depending on demand, and is always available and resilient to failures while optimizing costs.
- Containers (such as Docker containers) can be used as computational units when designing cloud native systems. They are more lightweight than virtual machines and provide portability, immutability, and flexibility.
- Dedicated platforms (such as Kubernetes) offer services to manage containers without directly handling the underlying layers. They provide container orchestration, cluster management, network services, and scheduling.
- Serverless computing is a model where the platform (such as Knative) manages servers and the underlying infrastructure, and the developer only focuses on the business logic. The backend functionality is enabled on a pay-per-use basis for cost optimization.
- A microservices architecture can be used to build cloud native applications, but it's not a requirement.
- To design cloud native applications, we'll use a service-based style characterized by services and their interactions.
- Cloud native services can be classified into application services (stateless) and data services (stateful).

---

1. P. Fremantle, "Cloud Native", *Paul Fremantle's Blog*, May 28, 2010, **http://mng.bz/Vy1G**.

2. Cloud Native Computing Foundation, "CNCF Cloud Native Definition v1.0," **http://mng.bz/de1w**.

3. NIST, "The NIST Definition of Cloud Computing," SP 800-145, September 2011, **http://mng.bz/rnWy**.

4. N.R. Herbst, S. Kounev, and R. Reussner, "Elasticity in Cloud Computing: What it is, and What it is Not," in *Proceedings of the 10th International Conference on Autonomic Computing (ICAC 2013)*, **http://mng.bz/BZm2**.

5. C. Davis, "Realizing Software Reliability in the Face of Infrastructure Instability," in *IEEE Cloud Computing* 4, 5, pp. 34-40, September/October 2017.

6. D.L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM* 15, 12 (December 1972), 1053-1058, **http://mng.bz/gwOl**.

7. J.E. Blyler, "Heuristics for resilience—A richer metric than reliability," *2016 IEEE International Symposium on Systems Engineering (ISSE)*, 2016, pp. 1-4.

8. A. Avižienis, J. Laprie, and B. Randell, "Fundamental Concepts of Dependability," 2001, **http://mng.bz/e7ez**.

9. A. Asta, "Observability at Twitter: technical overview, part 1," March 18, 2016, **http://mng.bz/pO8G**.

10. M. Fowler, "Infrastructure As Code," March 1, 2016, **https://martinfowler.com/bliki/InfrastructureAsCode.html**.

11. M. Fowler, "Continuous Delivery," May 30, 2013, **http://mng.bz/lRWo**.

12. M. Fowler, "Continuous Integration Certification," January 18, 2017, **http://mng.bz/xM4X**.

13. K. Mugrage, "My definition of DevOps," December 8, 2020, **http://mng.bz/AVox**.

14. J. Barr, "ACM Queue: Interview with Amazon's Werner Vogels," *AWS News Blog*, May 16, 2006, **http://mng .bz/ZpqA**.

15. N. Kratzke and R. Peinl, "ClouNS—a Cloud-Native Application Reference Model for Enterprise Architects," *2016 IEEE 20th International Enterprise Distributed Object Computing Workshop (EDOCW)*, 2016, pp. 1-10, doi: 10.1109/EDOCW.2016.7584353.

16. C. Sridharan, May 15, 2022, **http://mng.bz/YG5N**.

Previous chapter

< Part 1 Cloud native fundamentals

Next chapter

2 Cloud native patterns and technologies >